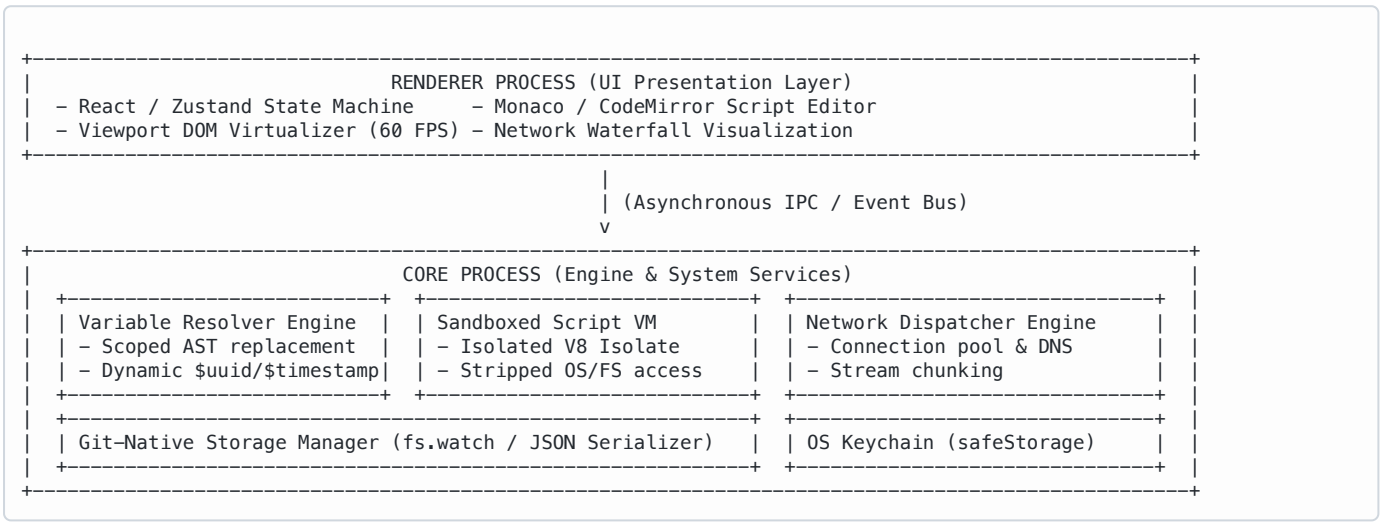
LLD & Schema Modeling of API Clients (Requestly)

Source: Arpit Bhayani Dissection • Systems Architecture
Focus: Multi-Process Topology, Schema Modeling & Data Entities

1. The Problem Space: Modern API Client Complexity

Modern developer API clients (Requestly, Postman, Insomnia, Hoppscotch) are no longer simple HTTP wrappers. They are local IDE-grade developer environments that must manage hierarchical request trees, resolve complex scoped variables, execute arbitrary user JavaScript test hooks, handle 50+ MB payloads without freezing the UI, and synchronize seamlessly with Git repositories without leaking secrets.

2. Multi-Process Architectural Topology



> **Process Decoupling Axiom:** The Renderer Process handles presentation exclusively. Heavy network I/O, script execution, file watching, and payload streaming reside in the Core Process to guarantee zero UI thread lockups.

3. Data Schema Modeling & Entity Relationships

The data domain is modeled around a clean, relational hierarchy serialized to Git-native JSON:

- **Workspace:** Top-level container mapped directly to a local Git directory root.
- **Collection & Folder:** Hierarchical recursive tree structure organizing related endpoints.
- **Request Entity:** Method, URL template, headers, query params, body type (form-data, JSON, binary), auth, and timeout settings.
- **Environment:** Key-value store defining global, workspace, and stage-specific runtime values.

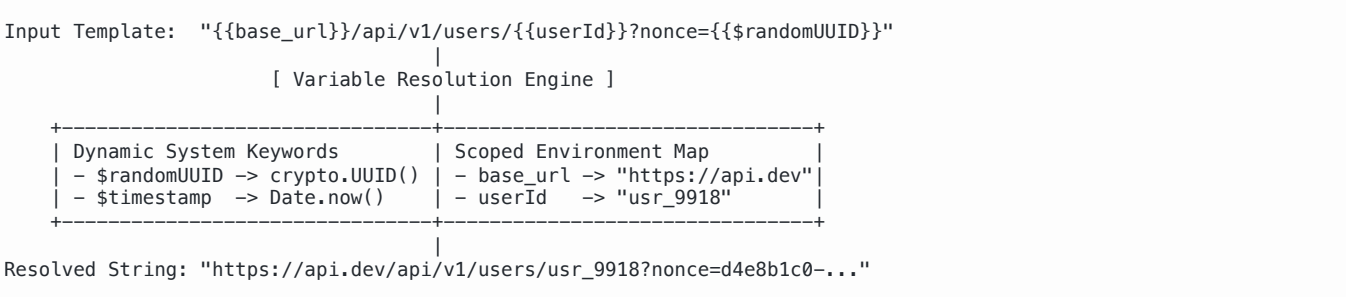
Variable Resolution & Sandboxed Script Execution

Source: Arpit Bhayani Dissection • Systems Architecture
Focus: Dynamic Templating, V8 Isolates & Security Sandboxing

4. Scoped Variable Resolution Engine

Before dispatching an HTTP call, the client parses all `{{variable_name}}` tags across URLs, headers, and request bodies. The engine enforces a strict precedence hierarchy:

> **Precedence Chain (Highest to Lowest):** $$ ext{Script Runtime Override} \succ ext{Folder / Collection Scope} \succ ext{Active Environment} \succ ext{Global Scope}$$



5. Sandboxed Script Execution Engine (Pre & Post Hooks)

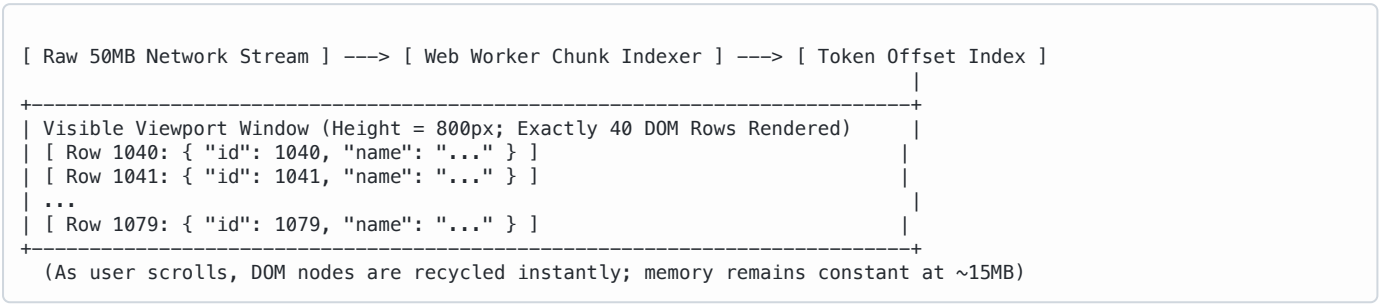
Users write custom JavaScript scripts to calculate cryptographic signatures (HMAC SHA-256) or assert test results. Running untrusted code on developer machines poses catastrophic security risks:

Execution Strategy	Mechanism	Security & Isolation	Evaluation
Naive `eval()`	Executes directly in main thread	Zero isolation; full access to app variables and DOM	Critical Vulnerability
Node.js `vm` module	Creates new V8 context	Can break out via prototype pollution into host `child_process`	High Risk
V8 Isolate (isolated-vm)	Completely separate V8 heap & memory	**100% isolated. Zero access to Node modules, fs, or network sockets**	**Production Standard**

> **The Sandbox Bridge:** Scripts communicate with the client only through an explicit, whitelisted bridge object (`rq.request`, `rq.environment`, `rq.test`). Script execution is constrained by a strict timeout (e.g. 5,000 ms) to prevent infinite loops.

6. Large Response Handling & Viewport Virtualization

When an API responds with a 50 MB JSON payload containing 500,000 records, standard web browsers freeze if all nodes are rendered into the DOM. Production clients solve this via **Viewport Window Virtualization**:



7. Git-Native Persistence & Secret Isolation

Modern developer platforms require Git-native workflows without proprietary cloud lock-in:

- **Deterministic JSON Files:** Workspaces and collections serialize to structured directories (`.api/collections/...`) with sorted JSON keys to guarantee clean Git diffs.
- **Secret Isolation:** Environment variables flagged as secret are *never* committed to Git. The client stores a placeholder in Git and persists the decrypted credential in the host OS Keychain via Electron's `safeStorage` API.
- **File System Watcher:** A background daemon (using `chokidar`) monitors workspace directories for external Git checkouts, updating the UI state reactively.

8. First-Principles System Design Takeaways

Design Dimension	Naive API Client Anti-Pattern	Production Architecture Standard
Concurrency	Single-threaded UI, network, and execution	Multi-process architecture isolating renderer, I/O, and scripts
Scripting	In-process script execution with full system rights	Secure V8 Isolate with explicit memory boundaries and CPU timeouts
Data Storage	Opaque SQLite database or proprietary cloud API	Git-native human-readable JSON files with filesystem watchers
Secret Hygiene	Plaintext tokens saved alongside collections	OS-level hardware-backed keychain encryption (safeStorage)